Model Deployment Pipeline

Machine Learning Assignment 15

October 7, 2026

1 Introduction

In data science, building a model is only half the battle; the model must ultimately be deployed so that other systems can use it to make predictions. This assignment closes the loop by taking the Logistic Regression classifier from Assignment 7 (Student Pass/Fail Classifier), saving it to disk, and loading it in a completely separate script to make live predictions without retraining.

2 The Deployment Pipeline

1. Saving the Model (`train_and_save.py`)

The first script handles the heavy lifting. It generates the historical student data, trains the Logistic Regression model, and then serializes the fitted model object to a binary file named `student_classifier.pkl` using the `joblib` library.

2. Loading and Predicting (`load_and_predict.py`)

The second script simulates a production environment (like a web server backend). It does not know how to train the model, nor does it have access to the original training data. It simply loads the pre-trained `.pkl` file into memory in milliseconds.

We passed three brand new student records through this loaded model:

- **Student 1** (9.0 Hrs, 95% Attendance): **Pass** (Confidence: 98.6%)
- **Student 2** (1.5 Hrs, 45% Attendance): **Fail** (Confidence: 0.4%)
- **Student 3** (5.0 Hrs, 60% Attendance): **Fail** (Confidence: 17.0%)

3 Error Handling in Production

A critical part of deployment is handling unexpected failures gracefully. If the `load_and_predict.py` script is executed before the model has been trained, the `student_classifier.pkl` file will be missing, causing a fatal `FileNotFoundError`.

Instead of letting the script crash with a messy, confusing Python traceback, we wrapped the loading mechanism in a `try/except` block. If the file is missing, the script catches the specific error and outputs a clean, human-readable instruction:

```
[ERROR] The model file 'student_classifier.pkl' was not found!
Please ensure you run 'train_and_save.py' first to generate the model file.
```

This ensures that the end-user (or the system administrator) knows exactly how to fix the problem without needing to debug the Python code itself.